

Tecnologia em Análise e Desenvolvimento de Sistemas - TADS

Estrutura de Dados

Prof. Luciano Vargas Gonçalves
E-mail: luciano.goncalves@riogrande.ifrs.edu.br



Estrutura da Dados

- **Aula 3**
 - **Estruturas com Struct**
 - **Programação em C**

Resumo – Aula 3

- **O que é uma struct?**
 - **É uma estrutura de dados heterogênea que permite agrupar variáveis de tipos diferentes (membros/atributos) sob um mesmo nome. Resolve o problema de gerenciar múltiplos vetores paralelos para um mesmo objeto do mundo real.**
- **Definição de Tipos (typedef):**
 - **O uso do typedef permite criar um "apelido" para a estrutura, dispensando o uso repetitivo da palavra-chave struct na declaração de variáveis.**

Resumo – Aula 3

- **Acesso aos Membros:**
 - **Por variável (estática):** Usa-se o operador ponto (.)
 - **Por ponteiro (dinâmica):** Usa-se o operador seta (->).
- **Passagem de struct para Funções:**
 - **Por Valor:** A função recebe uma cópia de toda a estrutura. Modificações feitas dentro da função não alteram a variável original.
- **Por Referência:**
 - **A função recebe o endereço de memória (ponteiro) da estrutura.** Modificações feitas através do ponteiro alteram diretamente a variável original no programa principal.

Resumo – Aula 3

Conceito	Passagem por Valor	Passagem por Referência
Sintaxe do Parâmetro	<code>void funcao(Pessoa p)</code>	<code>void funcao(Pessoa *p)</code>
Operador de Acesso	Ponto (<code>p.nome</code>)	Seta (<code>p->nome</code>)
Custo de Memória	Alto (duplica a estrutura na pilha)	Baixo (copia apenas o endereço do ponteiro)
Efeito nas Alterações	Alterações ficam apenas no escopo da função	Alterações permanecem no programa principal

Resumo – Aula 3 - Exemplo

- Exemplo estrutura Pessoa

```
1  #include <stdio.h>
2  #include <string.h>
3
4  // 1. Definição da estrutura com typedef
5  typedef struct
6  {
7      int cod;
8      char nome[30];
9      int idade;
10     char telefone[15];
11 } Pessoa;
```

Resumo – Aula 3 - Exemplo

```
13 // 2. Função com Passagem por REFERÊNCIA (Pessoa *ps)
14 // Permite alterar diretamente os dados da variável original no main.
15 // Utiliza o operador seta (->) para acessar os membros através do ponteiro.
16 void lerDadosPessoa(Pessoa *ps, int cod)
17 {
18     ps->cod = cod;
19
20     printf("Digite o nome: ");
21     scanf("%[^\n]", ps->nome); // Lê a string até a quebra de linha
22
23     printf("Digite a idade: ");
24     scanf("%d", &ps->idade);
25
26     printf("Digite o telefone: ");
27     scanf("%[^\n]", ps->telefone);
28 }
```

```
1  #include <stdio.h>
2  #include <string.h>
3
4  // 1. Definição da estrutura
5  typedef struct
6  {
7      int cod;
8      char nome[30];
9      int idade;
10     char telefone[15];
11 } Pessoa;
```

Resumo – Aula 3 - Exemplo

```
30 // 3. Função com Passagem por VALOR (Pessoa ps)
31 // Recebe apenas uma CÓPIA da estrutura.
32 // Utiliza o operador ponto (.) para acessar os membros.
33 void mostrarDadosPessoa(Pessoa ps)
34 {
35     printf("\n--- DADOS DA PESSOA ---\n");
36     printf("Código: %d\n", ps.cod);
37     printf("Nome: %s\n", ps.nome);
38     printf("Idade: %d anos\n", ps.idade);
39     printf("Telefone: %s\n", ps.telefone);
40     printf("-----\n");
41 }
```

```
1  #include <stdio.h>
2  #include <string.h>
3
4  // 1. Definição da estrutura
5  typedef struct
6  {
7      int cod;
8      char nome[30];
9      int idade;
10     char telefone[15];
11 } Pessoa;
```

Resumo – Aula 3 - Exemplo

```
43 int main()
44 {
45     // Declarando uma variável do tipo Pessoa
46     Pessoa p1;
47
48     // Passagem por REFERÊNCIA: enviamos o endereço (&p1) para que a função preencha a estrutura
49     lerDadosPessoa(&p1, 1);
50
51     // Passagem por VALOR: enviamos uma cópia de p1 para exibição
52     mostrarDadosPessoa(p1);
53
54     return 0;
55 }
```

```
1  #include <stdio.h>
2  #include <string.h>
3
4  // 1. Definição da estrutura
5  typedef struct
6  {
7      int cod;
8      char nome[30];
9      int idade;
10     char telefone[15];
11 } Pessoa;
```

Exercício 1

- Defina uma estrutura para armazenar dados de um apartamento:
 - Ex: Nome do condomínio, número, andar, quantidade cômodos, valor do aluguel, valor condomínio, box e etc.

```
5  typedef struct apartamento{  
6      char condomio [30];  
7      int num, andar, qtd_comodos,box;  
8      double v_aluguel, v_condominio;  
9  }Apartamento;
```

- Crie dois apartamentos, atribua valores aos seus membros (ap101, ap201), no main() e preencha com valores quais queres

Exercício 2

- Seguindo com a estrutura para armazenar dados de um apartamento:

```
5  typedef struct apartamento{  
6      char condomio [30];  
7      int num, andar, qtd_comodos,box;  
8      double v_aluguel, v_condominio;  
9  }Apartamento;
```

- Crie uma função para imprimir os valores de um apartamento (passagem por valor);

```
void mostraDadosApartamento(Apartamento ap){
```

- Apresente os dados dos apartamentos (ap101 e ap201)

Exercício 3

- Seguindo com a estrutura para armazenar dados de um apartamento:

```
5 typedef struct apartamento{
6     char condomio [30];
7     int num, andar, qtd_comodos,box;
8     double v_aluguel, v_condominio;
9 }Apartamento;
```

- Crie uma função para alterar o valor do aluguel de um apartamento (passagem por referência);

```
void alteraAluguelEm10porcentos(Apartamento *ap){
```

- Troque o valor do aluguel do ap201, para 1500.00R\$
- Mostre os dados do apartamento com o valor alterado.

Exercício 1 à 3

- Saída do Exercício 1

Apartamento:

Residencial: Avenida Brasil N°= 101 Andar= 1 Box= 10

Aluguel: 300.500000 e Condomínio 93.000000

Apartamento:

Residencial: Avenida Brasil N°= 201 Andar= 2 Box= 20

Aluguel: 300.330000 e Condomínio 100.000000

Apartamento:

Residencial: Avenida Brasil N°= 101 Andar= 1 Box= 10

Aluguel: 330.550000 e Condomínio 103.000000

—



Com aumento de 10%

Exercício 4

- Seguindo com a estrutura para armazenar dados de um apartamento:
 - Crie uma função para preencher os dados de um apartamento (passagem por referência);

```
void cadastraApartamento(Apartamento *ap)
{
    ""
    " Alocar memoria para um apartamento e retorna estrutura Apartamento"
    "";
    ap = (Apartamento *)malloc(sizeof(Apartamento));
    printf("\nInforme o nome do Condomínio");
    scanf("%s", ap->condominio);
}
```

Exercício 5

- Crie dois ponteiros de apartamentos no main(), utilize a função cadastra apartamento. Após, atribua valores aos campos dos apartamentos (ex. ap301 e ap401).
 - Use a função

```
void cadastraApartamento(Apartamento *ap)
```
 - Após mostre os dados apartamentos criados (ap301 e ap401);

Exercício 6

- Defina uma função que recebe um apartamento e retorna o box do apartamento;

```
int retornaBoxApartamento(Apartamento ap){
```

- Mostre os boxes dos apartamentos do 201 e 301;

Exercício 7

- Defina um função para criar um novo apartamento, a função recebe por parâmetro os dados do apartamento (Condomínio, andar, número, etc) e retorna o ponteiro do novo apartamento (atribua ao 401); Retorno por valor.

```
Apartamento criaNovoApartamento(char *condominio, int num, int andar, double v_aluguel, double v_condominio, int box){
```

- Mostre os dados do apartamento 202, 302 e 402 criados;

Exercício 8

- Defina a estrutura proprietário:

```
typedef struct
{
    char nome[30];
    char cpf[14];
} Proprietario;
```

- Implemente a função mostraProprietario;

```
void mostraDadosProprietario(Proprietario proprietario);
```

- Apresente todos os dados de um proprietário de um apartamento;
- Receba como entrada um proprietário

Exercício 9

- Redefina a estrutura apartamento para:



```
typedef struct
{
    char __condominio[30];
    int num, andar, qtd_comodos, box;
    double v_aluguel, v_condominio;
    Proprietario *dono;
    Data dtCompra;
} Apartamento;
```

```
typedef struct
{
    int dia, mes, ano;
} Data;
```

- Para incluir um proprietário para o apartamento;
- Também defina uma struct para armazenar a data de compra do apartamento

Exercício 10

- Defina dois proprietários fictícios no main(), um como variável e outro como ponteiro use o Malloc no último para reservar memória.
 - Preencha os dados dos proprietários
 - Atribua aos apartamentos a cada apartamento criado um proprietário

Exercício 3 - Apartamentos

- Crie a estrutura proprietário, com nome e cpf como membros;
- Adicione a estrutura Apartamento o membro, Proprietário;

```
typedef struct proprietario{  
    char nome [30];  
    char cpf [14];  
}Proprietario;
```

Qual a diferença entre
proprietário Ponteiro e Variável?

```
typedef struct apartamento{  
    char condomio [30];  
    int num, andar, qtd_comodos,box;  
    double v_aluguel, v_condominio;  
    Endereco *ed;  
    Proprietario *dono;  
}Apartamento;
```



Ponteiro para Proprietário

Exercício 4 - Apartamentos

- Defina uma **Struct Data**, com dia, mês e Ano de membro;
- Insira na **Struct Apartamento** a data de compra;

```
5  typedef struct proprietario{
6      char nome [30];
7      char cpf [14];
8  }Proprietario;
9
10 typedef struct data{
11     int dia,mes,ano;
12 }Data;
13
14 typedef struct apartamento{
15     char condomio [30];
16     int num, andar, qtd_comodos,box;
17     double v_aluguel, v_condominio;
18     Proprietario *dono;
19     Data compra;
20 }Apartamento;
```

Exercício 11 - Apartamentos

- Crie um função para vender um apartamento, que recebe um apartamento, um ponteiro para proprietário (*pr - comprador) que se tornará o novo dono, e um data:

```
void venderApartamento(Apartamento *ap, Proprietario *pr, Data dt){
```

- Atualize a função para mostrar todos os detalhes do apartamento;

```
void mostraDadosApartamento(Apartamento ap){
```

Exercício 5 - Apartamentos

- Crie um bloco de apartamentos com 8 apartamentos (vetor de apartamentos), defina os atributos de cada apartamento;
- Imprima o bloco de apartamentos com todos os dados dos apartamentos;
- Associe com a estrutura apartamento, um proprietário (pessoa) com a data de compra. Defina um mesmo proprietário para mais de um apartamento;



Dúvidas ??